Student Performance Analysis System

A Python-Based Project Report with Source Code and Output

Submitted by _____

Roll / Enrollment No. _____

Course / Semester _____

College _____

Guide / Faculty _____

Academic Year _____

Technologies: Python 3 | Pandas | NumPy | Matplotlib | Seaborn

1. Abstract

The Student Performance Analysis System is a menu-driven Python application that reads student marks from a CSV file, cleans the data, computes total, average, grade, pass/fail result and rank for every student, performs statistical analysis, generates charts and exports a performance report. It replaces manual result calculation with an automated, accurate and repeatable process and gives teachers a quick view of class performance, subject difficulty and individual standing.

2. Objectives

- Load student marks from a CSV file with validation of required columns.
- Clean the dataset: remove duplicates, and fix missing, non-numeric and out-of-range marks.
- Calculate total, average, grade, result (Pass/Fail) and rank.
- Perform subject-wise and class-wise statistical analysis.
- Generate charts and export CSV and text reports through a simple menu.

3. Tools and Technologies

Tool / Library	Purpose
Python 3	Core programming language
Pandas	CSV reading, cleaning, calculations
NumPy	Numeric operations and conditional logic
Matplotlib	Creating and saving plots
Seaborn	Styled statistical charts
pathlib	Portable file and folder paths

4. Methodology

- **Load and validate:** check the file exists, read it safely, and confirm Student_ID, Name and at least one subject column.
- **Clean:** drop duplicate rows; convert marks to numbers; treat values below 0 or above 100 as invalid; fill missing or invalid marks with the subject median; replace blank names with 'Unknown'.
- **Calculate:** Total = sum of marks; Average = mean of marks (2 decimals); Result = Pass only if every subject mark is at least 35; Rank by average (equal averages share a rank).
- **Analyse and report:** subject statistics, pass percentage, highest and lowest performer, four charts, CSV and text reports.

Average marks	Grade
90 and above	A+
80 to below 90	A
70 to below 80	B
60 to below 70	C
50 to below 60	D
Below 50	F

5. Source Code (main.py)

The complete program is listed below with line numbers. Subject columns are detected automatically, so the program works for any number of subjects.

```
 1  from pathlib import Path
 2  import pandas as pd
 3  import numpy as np
 4  import matplotlib.pyplot as plt
 5  import seaborn as sns
 6
 7  BASE_DIR = Path(__file__).resolve().parent
 8  DATA_FILE = BASE_DIR / "data" / "students.csv"
 9  CHART_DIR = BASE_DIR / "charts"
10  REPORT_DIR = BASE_DIR / "reports"
11
12  ID_COLUMN = "Student_ID"
13  NAME_COLUMN = "Name"
14  PASS_MARKS = 35
15
16  CHART_DIR.mkdir(exist_ok=True)
17  REPORT_DIR.mkdir(exist_ok=True)
18
19
20  def load_data():
21      """Load student records from the CSV file."""
22      if not DATA_FILE.exists():
23          print(f"Error: Dataset not found at {DATA_FILE}")
24          return None
25
26      try:
27          df = pd.read_csv(DATA_FILE)
28      except Exception as error:
29          print(f"Could not read the CSV file: {error}")
30          return None
31
32      required = {ID_COLUMN, NAME_COLUMN}
33      if not required.issubset(df.columns):
34          print("Error: CSV must contain Student_ID and Name columns.")
35          return None
36
37      subject_columns = [
38          col for col in df.columns
39          if col not in [ID_COLUMN, NAME_COLUMN]
40      ]
41      if not subject_columns:
42          print("Error: Add at least one subject column with marks.")
43          return None
44
45      return df
46
47
48  def get_subject_columns(df):
49      return [col for col in df.columns if col not in [ID_COLUMN, NAME_COLUMN]]
50
51
52  def clean_data(df):
53      """Remove duplicates and handle missing or invalid marks."""
54      cleaned = df.copy()
55      before_rows = len(cleaned)
56
57      cleaned = cleaned.drop_duplicates()
58      duplicate_count = before_rows - len(cleaned)
59
60      subjects = get_subject_columns(cleaned)
61      for subject in subjects:
62          cleaned[subject] = pd.to_numeric(cleaned[subject], errors="coerce")
63          invalid_mask = (cleaned[subject] < 0) | (cleaned[subject] > 100)
64          cleaned.loc[invalid_mask, subject] = np.nan
65
66          if cleaned[subject].isna().any():
67              median_value = cleaned[subject].median()
68              if pd.isna(median_value):
69                  median_value = 0
```

```
            cleaned[subject] = cleaned[subject].fillna(median_value)

    cleaned[NAME_COLUMN] = cleaned[NAME_COLUMN].fillna("Unknown").astype(str).str.strip()
    cleaned[NAME_COLUMN] = cleaned[NAME_COLUMN].replace("", "Unknown")
    cleaned = cleaned.dropna(subset=[ID_COLUMN])
    cleaned = cleaned.reset_index(drop=True)

    print("\nData cleaning completed.")
    print(f"Duplicate rows removed: {duplicate_count}")
    print(f"Rows remaining: {len(cleaned)}")
    print("Missing and invalid subject marks were replaced with the subject median.")
    return cleaned


def calculate_results(df):
    """Add total, average, grade, result, and rank columns."""
    results = df.copy()
    subjects = get_subject_columns(results)

    for subject in subjects:
        results[subject] = pd.to_numeric(results[subject], errors="coerce")

    results["Total"] = results[subjects].sum(axis=1)
    results["Average"] = results[subjects].mean(axis=1).round(2)
    results["Grade"] = results["Average"].apply(calculate_grade)

    # A student passes only when every subject mark is at least PASS_MARKS.
    results["Result"] = np.where(
        (results[subjects] >= PASS_MARKS).all(axis=1), "Pass", "Fail"
    )
    results["Rank"] = results["Average"].rank(
        method="min", ascending=False
    ).astype(int)

    return results.sort_values(
        ["Rank", NAME_COLUMN], ascending=[True, True]
    ).reset_index(drop=True)


def calculate_grade(average):
    if average >= 90:
        return "A+"
    if average >= 80:
        return "A"
    if average >= 70:
        return "B"
    if average >= 60:
        return "C"
    if average >= 50:
        return "D"
    return "F"


def display_data(df):
    print("\nSTUDENT DATA")
    print(df.to_string(index=False))


def statistical_analysis(df):
    subjects = get_subject_columns(df)
    marks = df[subjects].apply(pd.to_numeric, errors="coerce")

    print("\nSTATISTICAL ANALYSIS")
    print("\nSubject-wise statistics:")
    print(marks.describe().loc[["mean", "50%", "min", "max", "std"]].round(2)
          .rename(index={"50%": "median"}).to_string())

    results = calculate_results(df)
    total_students = len(results)
    passed = int((results["Result"] == "Pass").sum())
    pass_percentage = (passed / total_students * 100) if total_students else 0

    print(f"\nTotal students: {total_students}")
    print(f"Passed students: {passed}")
    print(f"Failed students: {total_students - passed}")
    print(f"Pass percentage: {pass_percentage:.2f}%")

```

```
    if not results.empty:
        print(f"Highest performer: {results.iloc[0][NAME_COLUMN]} "
              f"(Average: {results.iloc[0]['Average']:.2f})")
        lowest = results.sort_values("Average").iloc[0]
        print(f"Lowest performer: {lowest[NAME_COLUMN]} "
              f"(Average: {lowest['Average']:.2f})")


def show_grades(df):
    results = calculate_results(df)
    print("\nGRADES")
    print(results[[ID_COLUMN, NAME_COLUMN, "Average", "Grade", "Result"]]
          .to_string(index=False))


def show_rankings(df):
    results = calculate_results(df)
    print("\nSTUDENT RANKINGS")
    print(results[[ "Rank", ID_COLUMN, NAME_COLUMN, "Total",
                    "Average", "Grade", "Result"]].to_string(index=False))


def generate_charts(df):
    subjects = get_subject_columns(df)
    results = calculate_results(df)
    sns.set_theme(style="whitegrid")

    # 1. Subject-wise average marks
    subject_means = df[subjects].apply(pd.to_numeric, errors="coerce").mean()
    plt.figure(figsize=(8, 5))
    sns.barplot(x=subject_means.index, y=subject_means.values)
    plt.title("Subject-wise Average Marks")
    plt.xlabel("Subjects")
    plt.ylabel("Average Marks")
    plt.ylim(0, 100)
    plt.tight_layout()
    plt.savefig(CHART_DIR / "subject_performance.png", dpi=150)
    plt.close()

    # 2. Grade distribution
    grade_order = ["A+", "A", "B", "C", "D", "F"]
    plt.figure(figsize=(8, 5))
    sns.countplot(data=results, x="Grade", order=grade_order)
    plt.title("Grade Distribution")
    plt.xlabel("Grade")
    plt.ylabel("Number of Students")
    plt.tight_layout()
    plt.savefig(CHART_DIR / "grade_distribution.png", dpi=150)
    plt.close()

    # 3. Student average comparison
    comparison = results.sort_values("Average", ascending=False)
    plt.figure(figsize=(max(9, len(comparison) * 0.55), 5))
    sns.barplot(data=comparison, x=NAME_COLUMN, y="Average")
    plt.title("Student Performance Comparison")
    plt.xlabel("Student")
    plt.ylabel("Average Marks")
    plt.ylim(0, 100)
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.savefig(CHART_DIR / "student_comparison.png", dpi=150)
    plt.close()

    # 4. Marks distribution across all subjects
    all_marks = df[subjects].apply(pd.to_numeric, errors="coerce").melt(
        var_name="Subject", value_name="Marks"
    )
    plt.figure(figsize=(8, 5))
    sns.histplot(data=all_marks, x="Marks", bins=10, kde=True)
    plt.title("Distribution of Marks")
    plt.xlabel("Marks")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.savefig(CHART_DIR / "marks_distribution.png", dpi=150)
    plt.close()

    print(f"\nCharts generated and saved in: {CHART_DIR}")
```

```
    for filename in [
        "subject_performance.png",
        "grade_distribution.png",
        "student_comparison.png",
        "marks_distribution.png",
    ]:
        print(f" - {filename}")


def generate_report(df):
    results = calculate_results(df)
    csv_path = REPORT_DIR / "student_performance_report.csv"
    results.to_csv(csv_path, index=False)

    subjects = get_subject_columns(df)
    marks = df[subjects].apply(pd.to_numeric, errors="coerce")
    total_students = len(results)
    passed = int((results["Result"] == "Pass").sum())
    failed = total_students - passed
    pass_percentage = passed / total_students * 100 if total_students else 0

    summary_path = REPORT_DIR / "summary_report.txt"
    with open(summary_path, "w", encoding="utf-8") as file:
        file.write("STUDENT PERFORMANCE ANALYSIS REPORT\n")
        file.write("=" * 40 + "\n\n")
        file.write(f"Total students: {total_students}\n")
        file.write(f"Passed students: {passed}\n")
        file.write(f"Failed students: {failed}\n")
        file.write(f"Pass percentage: {pass_percentage:.2f}%\n\n")
        file.write("SUBJECT-WISE AVERAGE MARKS\n")
        file.write("-" * 28 + "\n")
        for subject, average in marks.mean().items():
            file.write(f"{subject}: {average:.2f}\n")

        if not results.empty:
            top = results.iloc[0]
            lowest = results.sort_values("Average").iloc[0]
            file.write("\nTOP PERFORMER\n")
            file.write(f"{top[NAME_COLUMN]} - Average: {top['Average']:.2f}\n")
            file.write("\nLOWEST PERFORMER\n")
            file.write(f"{lowest[NAME_COLUMN]} - Average: {lowest['Average']:.2f}\n")

        file.write("\nSTUDENT RANKINGS\n")
        file.write(results[["Rank", NAME_COLUMN, "Average", "Grade", "Result"]]
                   .to_string(index=False))
        file.write("\n")

    print("\nReports generated successfully:")
    print(f" - {csv_path}")
    print(f" - {summary_path}")


def print_menu():
    print("\n" + "=" * 46)
    print("     STUDENT PERFORMANCE ANALYSIS SYSTEM")
    print("=" * 46)
    print("1. Display Student Data")
    print("2. Clean Dataset")
    print("3. Perform Statistical Analysis")
    print("4. Calculate Grades")
    print("5. Display Student Rankings")
    print("6. Generate Charts")
    print("7. Generate Performance Report")
    print("8. Exit")
    print("=" * 46)


def main():
    df = load_data()
    if df is None:
        return

    cleaned_df = None

    while True:
        print_menu()
        choice = input("Enter your choice (1-8): ").strip()
```

```
301
302         if choice == "1":
303             display_data(cleaned_df if cleaned_df is not None else df)
304         elif choice == "2":
305             cleaned_df = clean_data(df)
306             df = cleaned_df
307         elif choice == "3":
308             statistical_analysis(cleaned_df if cleaned_df is not None else df)
309         elif choice == "4":
310             show_grades(cleaned_df if cleaned_df is not None else df)
311         elif choice == "5":
312             show_rankings(cleaned_df if cleaned_df is not None else df)
313         elif choice == "6":
314             generate_charts(cleaned_df if cleaned_df is not None else df)
315         elif choice == "7":
316             generate_report(cleaned_df if cleaned_df is not None else df)
317         elif choice == "8":
318             print("Thank you for using the Student Performance Analysis System!")
319             break
320         else:
321             print("Invalid choice. Please enter a number from 1 to 8.")
322
323
324 if __name__ == "__main__":
325     main()
```

6. Input Dataset (data/students.csv)

The following sample dataset was used to run the program. It deliberately contains data problems so that the cleaning step can be demonstrated: a **duplicate row** (S013), a **missing mark** (S009 English), an **invalid mark** (S010 Science = 150) and a **missing name** (S012).

```
Student_ID,Name,Maths,Science,English,Computer,History
S001,Aarav Patel,92,88,85,95,90
S002,Diya Shah,78,82,74,88,80
S003,Rohan Mehta,65,58,70,72,60
S004,Isha Desai,88,91,94,86,89
S005,Kabir Joshi,45,38,52,60,48
S006,Meera Trivedi,72,68,75,70,66
S007,Neel Parmar,30,42,55,48,50
S008,Priya Solanki,95,97,92,98,94
S009,Yash Vyas,58,62,,67,55
S010,Anika Rana,81,150,79,84,77
S011,Harsh Pandya,67,71,63,69,72
S012,,54,49,58,61,52
S013,Tara Bhatt,84,79,88,81,86
S014,Vivaan Gandhi,39,35,41,44,37
S015,Zoya Khan,76,80,73,78,82
S013,Tara Bhatt,84,79,88,81,86
```

7. Program Output

All output below was produced by running the program on the dataset above.

7.1 Main menu

```
==============================================
     STUDENT PERFORMANCE ANALYSIS SYSTEM
==============================================
1. Display Student Data
2. Clean Dataset
3. Perform Statistical Analysis
4. Calculate Grades
5. Display Student Rankings
6. Generate Charts
7. Generate Performance Report
8. Exit
==============================================
```

7.2 Option 1 - Display Student Data (raw data)

The raw data shows the duplicate S013 row, the missing English mark (NaN) for S009 and the missing name (NaN) for S012.

```
STUDENT DATA
Student_ID          Name  Maths  Science  English  Computer  History
      S001   Aarav Patel     92       88     85.0        95       90
      S002     Diya Shah     78       82     74.0        88       80
      S003   Rohan Mehta     65       58     70.0        72       60
      S004    Isha Desai     88       91     94.0        86       89
      S005   Kabir Joshi     45       38     52.0        60       48
      S006 Meera Trivedi     72       68     75.0        70       66
      S007   Neel Parmar     30       42     55.0        48       50
      S008 Priya Solanki     95       97     92.0        98       94
      S009     Yash Vyas     58       62      NaN        67       55
      S010    Anika Rana     81      150     79.0        84       77
      S011  Harsh Pandya     67       71     63.0        69       72
      S012           NaN     54       49     58.0        61       52
      S013    Tara Bhatt     84       79     88.0        81       86
      S014 Vivaan Gandhi     39       35     41.0        44       37
      S015     Zoya Khan     76       80     73.0        78       82
      S013    Tara Bhatt     84       79     88.0        81       86
```

7.3 Option 2 - Clean Dataset

```
Data cleaning completed.
Duplicate rows removed: 1
Rows remaining: 15
Missing and invalid subject marks were replaced with the subject median.
```

7.4 Option 1 again - Student Data after cleaning

S009's missing English mark became 73.5 and S010's invalid Science mark (150) became 69.5 (the subject medians); S012's name became 'Unknown'.

```
STUDENT DATA
Student_ID          Name  Maths  Science  English  Computer  History
      S001   Aarav Patel   92.0     88.0     85.0      95.0     90.0
      S002     Diya Shah   78.0     82.0     74.0      88.0     80.0
      S003   Rohan Mehta   65.0     58.0     70.0      72.0     60.0
      S004    Isha Desai   88.0     91.0     94.0      86.0     89.0
      S005   Kabir Joshi   45.0     38.0     52.0      60.0     48.0
      S006 Meera Trivedi   72.0     68.0     75.0      70.0     66.0
      S007   Neel Parmar   30.0     42.0     55.0      48.0     50.0
      S008 Priya Solanki   95.0     97.0     92.0      98.0     94.0
      S009     Yash Vyas   58.0     62.0     73.5      67.0     55.0
      S010    Anika Rana   81.0     69.5     79.0      84.0     77.0
      S011  Harsh Pandya   67.0     71.0     63.0      69.0     72.0
      S012       Unknown   54.0     49.0     58.0      61.0     52.0
      S013    Tara Bhatt   84.0     79.0     88.0      81.0     86.0
      S014 Vivaan Gandhi   39.0     35.0     41.0      44.0     37.0
      S015     Zoya Khan   76.0     80.0     73.0      78.0     82.0
```

7.5 Option 3 - Statistical Analysis

```
STATISTICAL ANALYSIS

Subject-wise statistics:
        Maths  Science  English  Computer  History
mean    68.27    67.30    71.50     73.40    69.20
median  72.00    69.50    73.50     72.00    72.00
min     30.00    35.00    41.00     44.00    37.00
max     95.00    97.00    94.00     98.00    94.00
std     19.69    19.63    15.36     15.93    17.97

Total students: 15
Passed students: 14
Failed students: 1
Pass percentage: 93.33%
Highest performer: Priya Solanki (Average: 95.20)
Lowest performer: Vivaan Gandhi (Average: 39.20)
```

7.6 Option 4 - Calculate Grades

```
GRADES
Student_ID          Name  Average Grade Result
      S008 Priya Solanki     95.2    A+   Pass
      S001   Aarav Patel     90.0    A+   Pass
      S004    Isha Desai     89.6     A   Pass
      S013    Tara Bhatt     83.6     A   Pass
      S002     Diya Shah     80.4     A   Pass
      S010    Anika Rana     78.1     B   Pass
      S015     Zoya Khan     77.8     B   Pass
      S006 Meera Trivedi     70.2     B   Pass
      S011  Harsh Pandya     68.4     C   Pass
      S003   Rohan Mehta     65.0     C   Pass
      S009     Yash Vyas     63.1     C   Pass
      S012       Unknown     54.8     D   Pass
      S005   Kabir Joshi     48.6     F   Pass
      S007   Neel Parmar     45.0     F   Fail
      S014 Vivaan Gandhi     39.2     F   Pass
```

7.7 Option 5 - Student Rankings

```
STUDENT RANKINGS
 Rank Student_ID          Name  Total  Average Grade Result
    1       S008 Priya Solanki  476.0     95.2    A+   Pass
    2       S001   Aarav Patel  450.0     90.0    A+   Pass
    3       S004    Isha Desai  448.0     89.6     A   Pass
    4       S013    Tara Bhatt  418.0     83.6     A   Pass
    5       S002     Diya Shah  402.0     80.4     A   Pass
    6       S010    Anika Rana  390.5     78.1     B   Pass
    7       S015     Zoya Khan  389.0     77.8     B   Pass
    8       S006 Meera Trivedi  351.0     70.2     B   Pass
    9       S011  Harsh Pandya  342.0     68.4     C   Pass
   10       S003   Rohan Mehta  325.0     65.0     C   Pass
   11       S009     Yash Vyas  315.5     63.1     C   Pass
   12       S012       Unknown  274.0     54.8     D   Pass
   13       S005   Kabir Joshi  243.0     48.6     F   Pass
   14       S007   Neel Parmar  225.0     45.0     F   Fail
   15       S014 Vivaan Gandhi  196.0     39.2     F   Pass
```

7.8 Option 6 - Generate Charts

```
Charts generated and saved in: charts
 - subject_performance.png
 - grade_distribution.png
 - student_comparison.png
 - marks_distribution.png
```

7.9 Option 7 - Generate Performance Report

```
Reports generated successfully:
 - reports/student_performance_report.csv
 - reports/summary_report.txt
```

8. Generated Charts

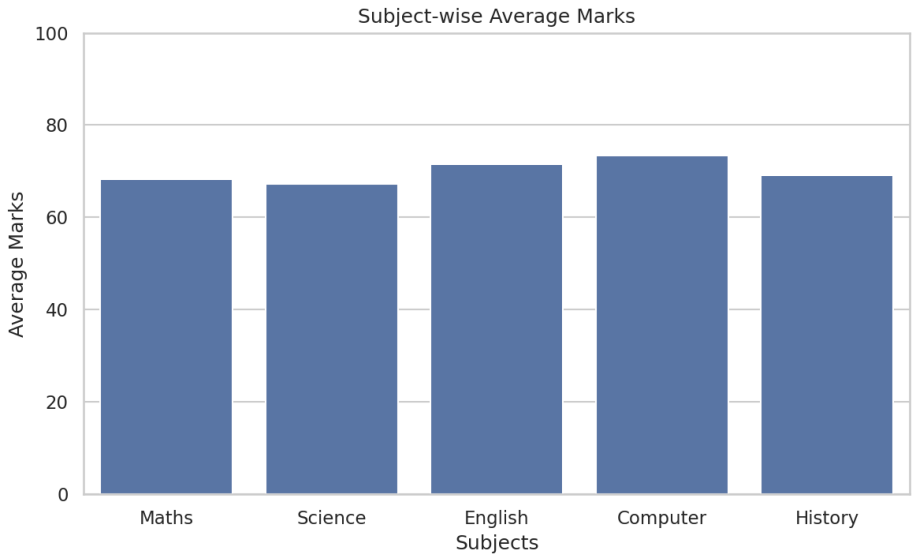


Figure 1: Subject-wise average marks

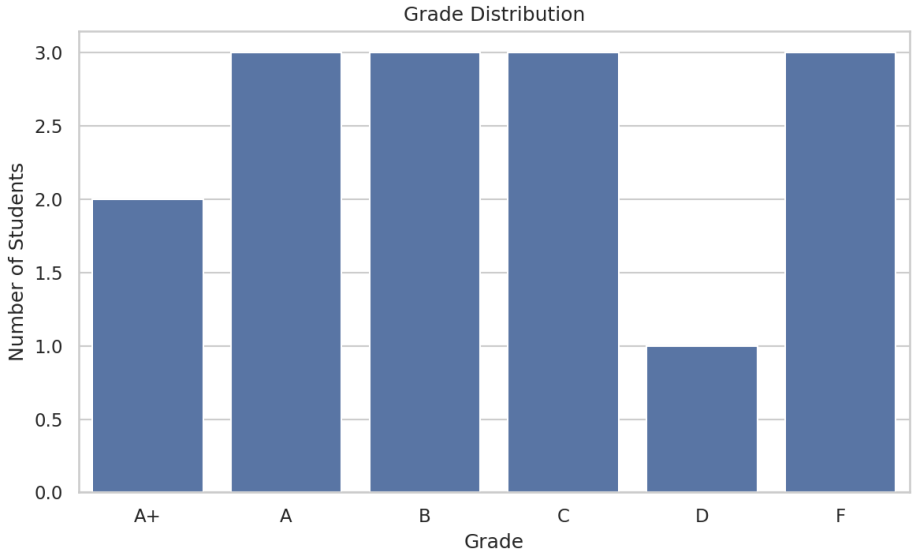


Figure 2: Grade distribution

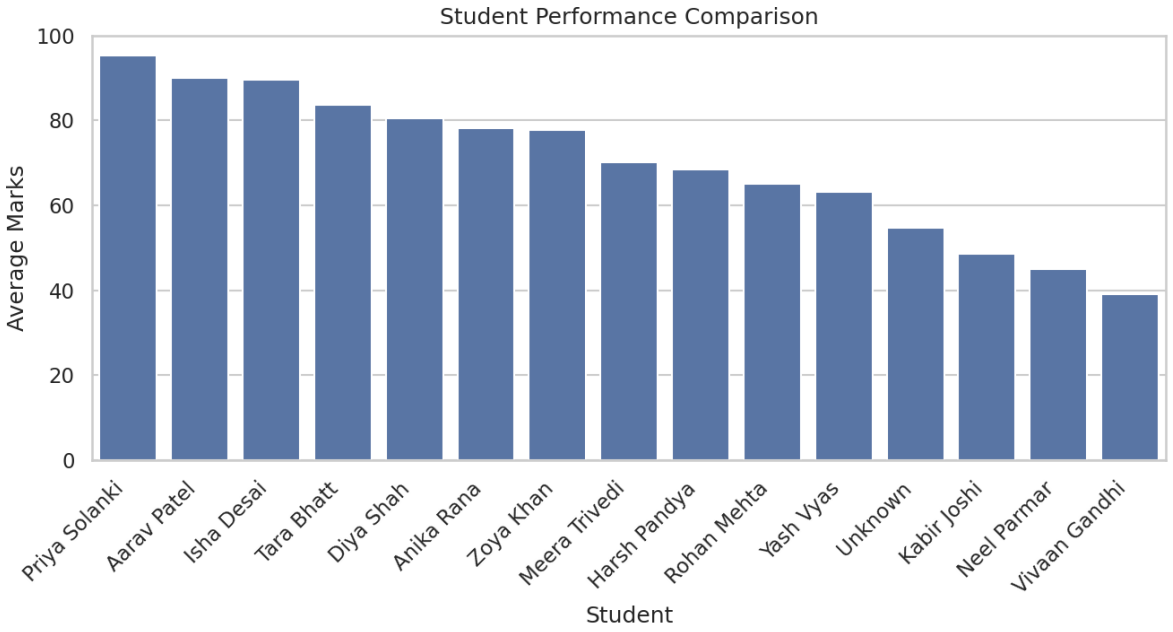


Figure 3: Student performance comparison

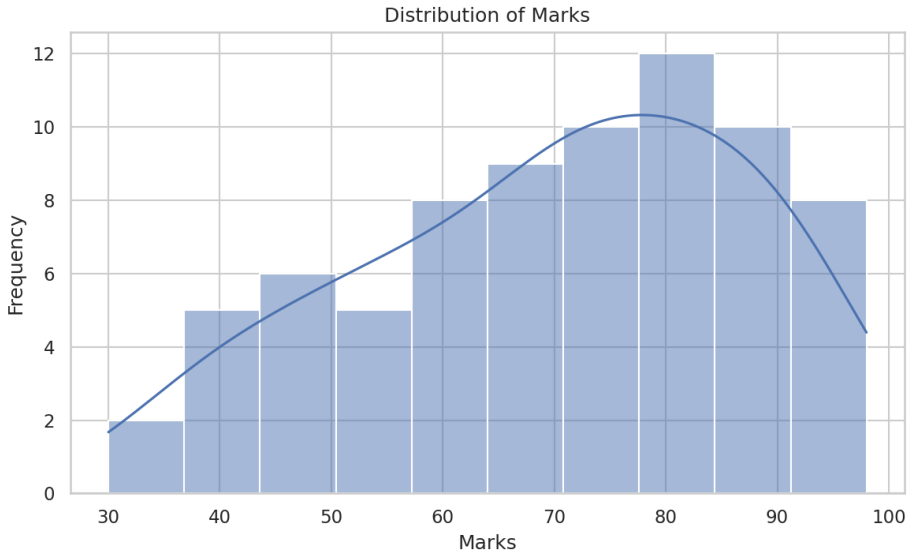


Figure 4: Distribution of marks

9. Exported Reports

9.1 summary_report.txt

```
STUDENT PERFORMANCE ANALYSIS REPORT
====================================

Total students: 15
Passed students: 14
Failed students: 1
Pass percentage: 93.33%

SUBJECT-WISE AVERAGE MARKS
---------------------------
Maths: 68.27
Science: 67.30
English: 71.50
Computer: 73.40
History: 69.20

TOP PERFORMER
Priya Solanki - Average: 95.20

LOWEST PERFORMER
Vivaan Gandhi - Average: 39.20

STUDENT RANKINGS
 Rank          Name  Average Grade Result
    1 Priya Solanki     95.2    A+   Pass
    2   Aarav Patel     90.0    A+   Pass
    3    Isha Desai     89.6     A   Pass
    4    Tara Bhatt     83.6     A   Pass
    5     Diya Shah     80.4     A   Pass
    6    Anika Rana     78.1     B   Pass
    7     Zoya Khan     77.8     B   Pass
    8 Meera Trivedi     70.2     B   Pass
    9  Harsh Pandya     68.4     C   Pass
   10   Rohan Mehta     65.0     C   Pass
   11     Yash Vyas     63.1     C   Pass
   12       Unknown     54.8     D   Pass
   13   Kabir Joshi     48.6     F   Pass
   14   Neel Parmar     45.0     F   Fail
   15 Vivaan Gandhi     39.2     F   Pass
```

The file `student_performance_report.csv` is also created in the reports folder. It contains every subject mark together with Total, Average, Grade, Result and Rank for each student.

10. Observations

- The cleaning step removed 1 duplicate row and replaced 2 bad marks and 1 blank name, leaving 15 valid student records.
- Priya Solanki ranked first with an average of 95.20; Vivaan Gandhi had the lowest average (39.20).
- Only Neel Parmar failed (Maths = 30, below the pass mark of 35). 14 of 15 students passed, a pass percentage of 93.33%.
- Kabir Joshi (average 48.60) and Vivaan Gandhi (average 39.20) received grade F yet show Result = Pass, because the grade is based on the average while the pass rule checks each subject against 35. An institution may wish to align the two rules.
- Computer had the highest class average (73.40) and Science the lowest (67.30).

11. Limitations and Future Scope

- Command-line interface only; the pass mark, grading scale and maximum marks are fixed in the code.

- Median replacement of missing marks is an assumption and may not match the actual marks.
- Future work: a graphical or web interface (Tkinter / Streamlit), Excel input, PDF report cards, configurable grading rules and machine-learning based prediction of at-risk students.

12. Conclusion

The system automates the complete result-processing cycle: it cleans raw marks, applies one consistent grading and pass rule, analyses performance statistically, visualises it through charts and exports ready-to-use reports. It reduces manual effort and calculation errors and can be extended to suit the needs of an institution.

13. References

- Python Documentation - https://docs.python.org/3/
- Pandas Documentation - https://pandas.pydata.org/docs/
- NumPy Documentation - https://numpy.org/doc/
- Matplotlib Documentation - https://matplotlib.org/stable/
- Seaborn Documentation - https://seaborn.pydata.org/